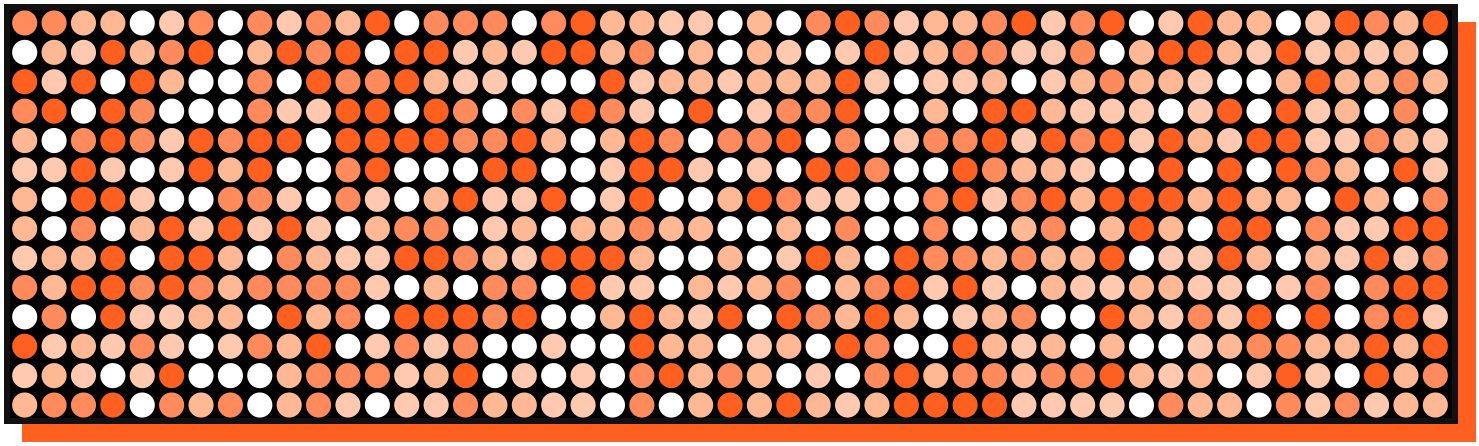
PAPYRUS EVIDENCE PACK

DEPLOYMENT REQUIREMENTS

Compute, storage, networking, runtime dependencies, offline constraints and external services.



RELEASE **0.1.0+fd55e38**	BUILT **2026-10-20T13:03:06Z**
IMAGE **ghcr.io/beaglabs/papyrus/papyrus**	INDEX DIGEST **sha256:b38facf4950270443ac39dc5bbaeb4ed5653341b61608526163e929a449ebd8a**

Prepared by Beag Labs from the Papyrus source and its published release image. Describes the software and the responsibilities it leaves to a deploying organization. It is not a certification, an authorization or a third-party assessment.

DEPLOYMENT REQUIREMENTS

This is an overview of what a Papyrus deployment needs from its host, network and identity provider. Papyrus makes no assumption about cloud provider, region, orchestrator or hosting model: it runs wherever an OCI container can run with a persistent volume. Choices such as the identity tenant, secret storage, certificate authority and network layout are made per deployment. The architecture document in the same evidence pack covers components and data flows.

DEPLOYMENT PROFILES

Every deployment runs one profile, set with `PAPYRUS_PROFILE`. The profile describes what Papyrus may assume about the network around it. It does not remove features.

PROFILE	INTENDED FOR	NETWORK ASSUMPTION
`commercial`	Connected commercial deployments (default)	Services the operator allows may be on the public internet
`government`	Connected government deployments	As `commercial`, but no commercial model endpoint is ever used by default
`disconnected`	Air-gapped and closed networks	Nothing outside the authorization boundary is assumed reachable

On the `disconnected` profile, every service Papyrus uses is an endpoint the operator provides inside the boundary. Papyrus does not contact Beag Labs, never uses public package registries, and has no default model endpoint. Identity, directory, models, packages, images and customer systems all work, against internal endpoints.

The identity cloud is selected separately from the profile, with `PAPYRUS_ENTRA_CLOUD` (`Public`, `USGov` or `USGovDoD`).

PLATFORM

ITEM	VALUE
Container images	`ghcr.io/beaglabs/papyrus/papyrus` and the DoD variant `ghcr.io/beaglabs/papyrus/papyrus-dod`; both can be mirrored into a customer registry
Architectures	`linux/amd64` and `linux/arm64`, published as one multi-architecture index
Signing	Each published index is signed with Sigstore Cosign and carries SLSA build provenance
Base image	Chainguard Wolfi (`cgr.dev/chainguard/wolfi-base`)
Runtime	Node.js 24, Python 3.12, busybox `sh`
User	Runs as the non-root `nonroot` user (UID 65532)
Host	Linux with the Landlock LSM enabled, on any OCI-compatible container runtime or orchestrator. The health check confirms that the `landstrip` workspace sandbox can enforce its write boundary
Privileges	No privileged mode and no added capabilities; runs under the runtime's default seccomp profile
Entrypoint and health	`node /entrypoint.mjs`; `HEALTHCHECK` runs `node /healthcheck.mjs` every 30 s

macOS is supported for local development only. On hosts where the sandbox cannot be applied, the daemon runs but native workspace execution is disabled.

The DoD image is the same runtime with two build-time defaults: `PAPYRUS_ENTRA_CLOUD=USGovDoD` and the offline license authority public key, so a signed license can be activated without contacting Beag Labs. Set `PAPYRUS_PROFILE` explicitly.

COMPUTE

Papyrus does not publish a fixed CPU or memory minimum, and it does not assume any particular instance type or platform. Size to workload. What drives sizing:

- Concurrent agent sessions, workflows and schedules running in the daemon.

- Document ingestion and local embedding (CPU-bound, in a worker thread with a 384 MB heap limit).
- Native workspace executions (Python, Pandoc, LibreOffice) in sandbox worker processes.
- Factory App builds in the Factory sidecar process.
- Uploads, which are limited to 64 MiB per file by default (`PAPYRUS_ATTACHMENT_MAX_MIB`).

GPU. No component requires a GPU. The Knowledge Index embedding model runs on CPU. Chat model inference runs at the model gateway the operator configures, not inside Papyrus.

STORAGE

Persistent volume. One writable volume at `PAPYRUS_DATA_DIR` (image default `/var/lib/papyrus`), owned by UID 65532. All state must persist across container restarts and replacements. Databases are LibSQL/SQLite files; no external database server is used.

PATH UNDER THE DATA DIRECTORY	CONTENTS
`agent.db`	Daemon state: configuration, policies, action ledger, audit, MCP servers and call log, Classes and records, workflows, App releases
`mastra.db`	Session threads and messages
`jobs.db`	Schedules and background jobs
`observability.db`	Trace spans and logs
`.agentfs/<id>.db`	AgentFS workspace and Library files
`secrets.db`	Encrypted model gateway credentials saved in the portal
`usage.db`	Hourly agent execution time
`factory/`	Factory sidecar database, session checkouts and its encrypted credentials
`models/nomic-embed-text-v1.5/`	Embedding model pack (downloaded or imported)
`skills/enabled/`	Approved agent skills
`bootstrap.json` and related state files	First-run configuration

Secrets and keys. Credentials Papyrus stores are encrypted at the application layer with AES-256-GCM. Key material is provided by the operator, as a 32-byte master key file mounted read-only (`/etc/papyrus/secrets/master.key` or `PAPYRUS_SECRET_KEY_FILE`). Papyrus can instead resolve credentials by reference from an external key management service; which service is used is decided per deployment. Model and connector credentials are always configured as references, never as inline values.

Certificates. TLS certificates are provided by the operator and can be issued by any certificate authority, including a private or local CA. TLS is terminated by the operator's proxy or by Papyrus itself (`PAPYRUS_TLS_CERT` , `PAPYRUS_TLS_KEY`). Private trust roots are supported for connections Papyrus makes: `PAPYRUS_TLS_CA` for appliance consoles and `PAPYRUS_PACKAGE_MIRROR_CA_FILE` for package mirrors.

Other mounts. `PAPYRUS_TOOLCHAIN_DIR` (default `/opt/papyrus/toolchain`) holds Pandoc and LibreOffice for document conversion tools; they are not included in the image.

Backups. Back up `agent.db` , `mastra.db` and `jobs.db` together, plus the rest of the data directory, with consistent SQLite backups. Keep the master key separately from database backups; losing it makes saved credentials unrecoverable.

NETWORKING

Listening ports.

PORT	BOUND BY	PROTOCOL	NOTES
`3210` (`PAPYRUS_PORT`)	Daemon	HTTP, or HTTPS when Papyrus terminates TLS	Portal, API, App content and public intake. First-run setup is served on the same port
`4199` (`PAPYRUS_FACTORY_PORT`) on `127.0.0.1`	Factory sidecar	HTTP	Inside the container only; reached through the daemon

Inbound.

- Persistent mode (`PAPYRUS_MODE=persistent`) requires an HTTPS public origin (`PAPYRUS_PUBLIC_ORIGIN`). The configured origin is authoritative; request headers cannot change sign-in redirect addresses.
- Users' browsers must reach that origin and, if one is set, the separate App origin (`PAPYRUS_APP_ORIGIN`).
- Public intake routes (`/c/` , `/r/` , `/hooks/`) need to be reachable only by the parties who submit forms, follow request links or send webhooks, and only when those features are used.
- Restrict access to the first-run setup page until identity configuration is complete; it does not require a setup token.

Network dependencies. These are the services Papyrus can connect to. None of them has to be on the public internet: each is reached at an address the operator configures, over whatever network path the operator provides. On the `disconnected` profile they are expected to sit inside the authorization boundary.

SERVICE	USED FOR	NEEDED	CONNECTED PROFILES	DISCONNECTED PROFILE
Identity provider (Microsoft Entra ID)	Sign-in and token validation	Yes, in persistent mode	Entra for the configured identity cloud	Reached inside the boundary; browsers and the daemon need a path to it
Directory (Microsoft Graph)	Staff directory; optional first-run role setup	No	Graph for the configured identity cloud	Reached inside the boundary when available; otherwise the directory lists people who have signed in
Model gateway	Agent model inference	For agent work	Any OpenAI-compatible endpoint the operator chooses	An endpoint inside the boundary; no default is assumed
Customer systems (MCP servers, Mainframe TN3270, SOAP services, workflow HTTP targets)	Integrations the customer connects	No	Operator's endpoints; internal hosts are allow-listed explicitly	Operator's endpoints inside the boundary
Package sources	Skill and session package installs	No	Internal mirrors, or public registries unless disabled (`PAPYRUS_PACKAGE_PUBLIC_REGISTRIES=false`)	Internal mirrors only; public registries are never used
Embedding model	Knowledge Index semantic search	No	Downloaded once from the model publisher, or imported	Imported from an offline pack, with downloads turned off (`PAPYRUS_EMBEDDING_DOWNLOAD=0`)
Source control	Factory App source	No	GitHub or a self-managed GitLab	A self-managed GitLab inside the boundary
Container registry	Install and update	At install and update	`ghcr.io` or a customer registry	A customer registry (`PAPYRUS_UPDATE_REPOSITORIES`)
Error reporting (Sentry)	Exception reports	No	Off unless a DSN is configured	Off unless an internal DSN is configured
Beag Labs services (license authority, control plane)	Registration and usage analytics; status, metering and brokered sign-in for deployments Beag Labs manages	No	Only for registered or Beag Labs-managed deployments; registration can be turned off (`PAPYRUS_BEAGLABS_REGISTRATION=0`)	Never contacted

Host-level network policy (firewalls, proxies, segmentation) remains the operator's control.

IDENTITY

Papyrus signs people in with Microsoft Entra ID; it keeps no local user accounts or passwords. A deployment uses the customer's own Entra app registration, configured in first-run setup or the environment (`PAPYRUS_ENTRA_TENANT_ID` , `PAPYRUS_ENTRA_CLIENT_ID`).

REQUIREMENT	DETAIL
Redirect URI (Web)	`https://<public-origin>/api/auth/entra/callback`
Exposed scope	`api://<client-id>/access_as_user` by default (`PAPYRUS_ENTRA_SCOPE` overrides)
Client secret	Optional; used for the confidential-client flow and the staff directory
Identity cloud	`PAPYRUS_ENTRA_CLOUD` = `Public` , `USGov` or `USGovDoD`
App roles	`Papyrus.System.Owner` , `Papyrus.Integration.View` , `Papyrus.Integration.Manage` , `Papyrus.Security.Manage` , `Papyrus.Action.Approve` , `Papyrus.Audit.View`
Role assignment	Roles are read from token claims; assign users or groups in Entra
First-run role setup (optional)	An Entra administrator signs in once with delegated permission to add the roles; Papyrus makes that administrator System Owner and does not store the token. The role manifest can be added by hand instead
Staff directory (optional)	Microsoft Graph application permission `User.Read.All` with admin consent

Deployments that Beag Labs manages can instead sign people in through Beag Labs' multi-tenant Entra app, with roles held in the deployment. Customers who cannot accept Beag Labs as an identity broker use their own app registration.

Published Factory Apps can use additional OIDC providers configured in `PAPYRUS_FACTORY_OIDC_PROVIDERS` .

RUNTIME DEPENDENCIES

Included in the image.

- Node.js 24 and the daemon's production dependencies, including the Mastra agent framework, LibSQL, AgentFS, the `landstrip` sandbox and the OpenAI-compatible model client.
- The built web portal.
- The Factory sidecar, started by default (`PAPYRUS_FACTORY_SIDECAR=1`).
- Python 3.12 with document-handling libraries and the TN3270 bridge.
- The embedding model provisioner, without model weights.
- No browser engine.

Provided by the operator.

DEPENDENCY	NEEDED FOR
Persistent volume	All state
Entra tenant and app registration	Sign-in
Signed license, or a Beag Labs-managed deployment's entitlement	Product entitlement
OpenAI-compatible model endpoint and credential	Agent work
TLS certificate, or a TLS-terminating proxy	Persistent mode
Master key file, or an external key management service	Encrypted credential storage
Embedding model pack	Knowledge Index semantic search
Pandoc and LibreOffice under `PAPYRUS_TOOLCHAIN_DIR`	Document conversion tools
Package mirrors	Skill and session package installs where public registries are not used

Entitlement. A signed license is verified locally against the license authority public keys configured in `PAPYRUS_LICENSE_AUTHORITIES` ; no call to Beag Labs is needed. Deployments Beag Labs manages receive their entitlement from the control plane instead.

DATA SENT TO BEAG LABS

Only registered or Beag Labs-managed deployments on a connected profile send anything to Beag Labs. A `disconnected` deployment sends nothing.

SERVICE	DATA	WHEN
License authority (`license.beaglabs.com`)	Registration details (deployment ID, public origin, version, Entra client ID, and the hosting platform's instance metadata where it provides any); usage analytics events with the signed-in person's Entra object ID, display name and tenant ID	Registered deployments, unless `PAPYRUS_BEAGLABS_REGISTRATION=0`
Control plane (`papyrus.beaglabs.com`)	Deployment credential, hourly agent execution seconds, sign-in broker state	Beag Labs-managed deployments only

Papyrus is not a cross-domain solution and makes no claim of GCC High, DoD IL4, IL6 or SIPR accreditation.